

Задание A1. Анализ строковых сортировок.

Номера посылок на CodeForces:

- A1m: 374537192
- A1q: 374593854
- A1r: 374726812
- A1rq: 374925935

В работе исследуется эффективность сортировки строк по двум критериям: количеству посимвольных сравнений и времени выполнения. Для анализа были выбраны стандартные и специализированные алгоритмы: Standard Quick Sort, Standard Merge Sort, ternary String QuickSort, String MergeSort с использованием длины наибольшего общего префикса, MSD Radix Sort без переключения на String QuickSort и гибридный MSD Radix Sort с переходом на String QuickSort для малых подмассивов. Реализация выполнена на языке C++.

Описание программной реализации

Для проведения эксперимента были реализованы два вспомогательных класса: StringGenerator и StringSortTester.

Класс StringGenerator предназначен для генерации тестовых массивов строк. В качестве алфавита используются 74 символа: заглавные и строчные латинские буквы, цифры и специальные символы, указанные в условии задания. Генератор позволяет создавать строки длиной от 10 до 200 символов. На его основе формируются три типа выборок: случайная, обратно отсортированная и почти отсортированная. Случайная выборка состоит из независимых случайных строк, обратно отсортированная получается путем сортировки массива и последующего разворота, почти отсортированная создается из предварительно отсортированного массива путем небольшого числа случайных перестановок.

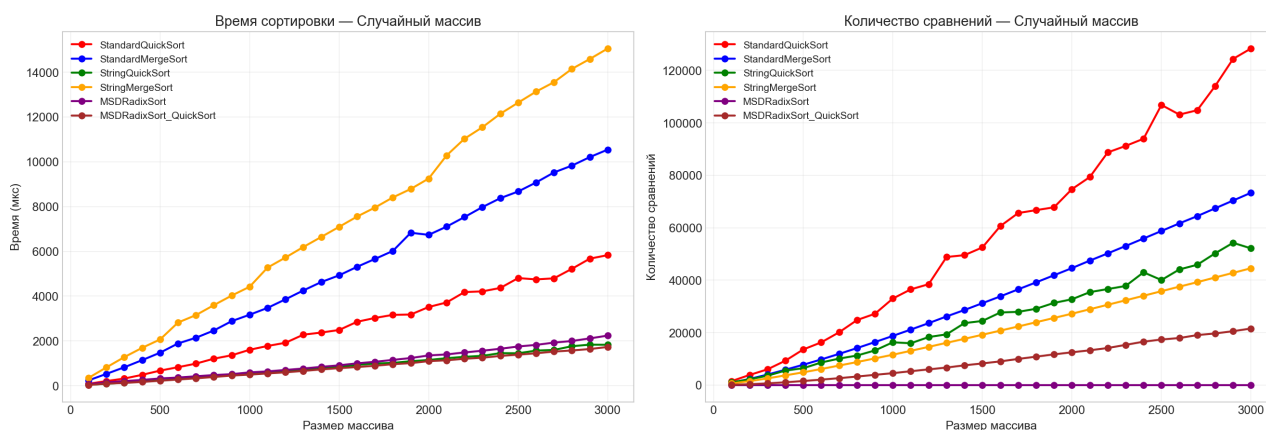
Для удобства эксперимента массивы максимального размера генерируются заранее, после чего из них берутся подмассивы нужной длины от 100 до 3000 элементов с шагом 100. Такой подход позволяет уменьшить влияние случайных отклонений и использовать одинаковую исходную базу данных для всех алгоритмов.

Класс StringSortTester отвечает за измерение времени работы алгоритмов сортировки и подсчет числа посимвольных сравнений. Перед каждым запуском счетчик сравнений сбрасывается, затем выполняется сортировка, после чего измеряется время работы в микросекундах. Посимвольные сравнения учитываются только в тех операциях, где действительно происходит сравнение символов строк, служебные операции и обмены элементов в статистику не входят.

В главной программе для каждого типа выборки и каждого размера массива сортировка выполняется несколько раз, после чего вычисляется среднее время выполнения и среднее число сравнений. Полученные значения сохраняются в CSV-файлы и используются для построения графиков, по которым проводится дальнейший анализ.

Результаты эксперимента

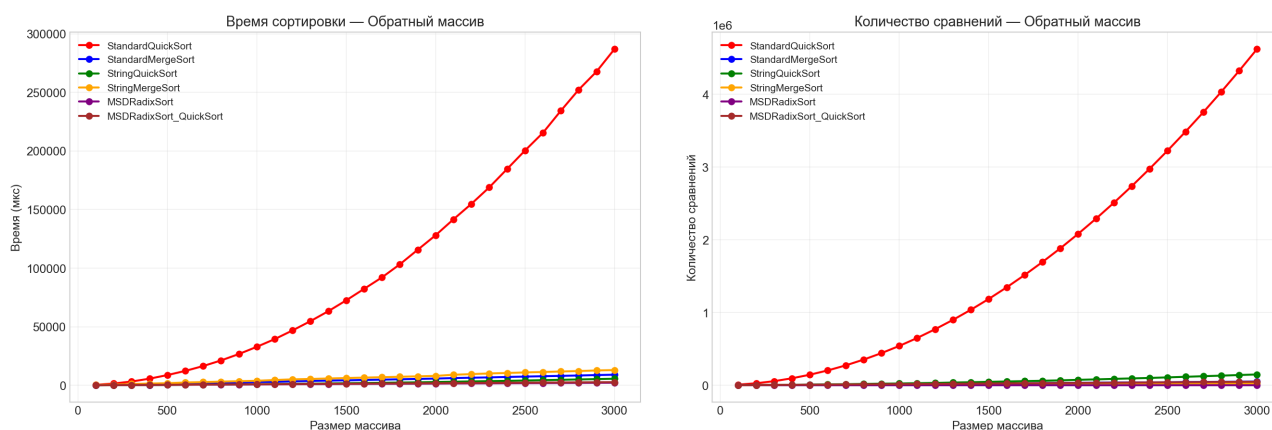
Случайный массив



На случайном массиве **StandardMergeSort** выполняется медленнее, чем **StandardQuickSort**, что особенно заметно при увеличении размера массива. Среди специализированных алгоритмов наибольшее время выполнения показал **StringMergeSort**: на больших размерах массива его отрыв от остальных методов становится особенно заметным. Это может быть связано с тем, что при сортировке строк MergeSort вынужден многократно выполнять посимвольные сравнения и обрабатывать общие префиксы, что увеличивает накладные расходы по сравнению с более удачными строковыми методами.

По количеству посимвольных сравнений стандартные алгоритмы оказались менее эффективными, чем специализированные. При этом **StandardQuickSort** выполняет больше сравнений, чем **StandardMergeSort**, а график для него имеет менее равномерный характер с заметными колебаниями. Среди специализированных алгоритмов лучше всего по числу сравнений показал себя **MSDRadixSort**: его значение остается существенно ниже, чем у остальных методов. Наименее удачный результат среди специализированных алгоритмов по числу сравнений на данном наборе данных показал **StringQuickSort**.

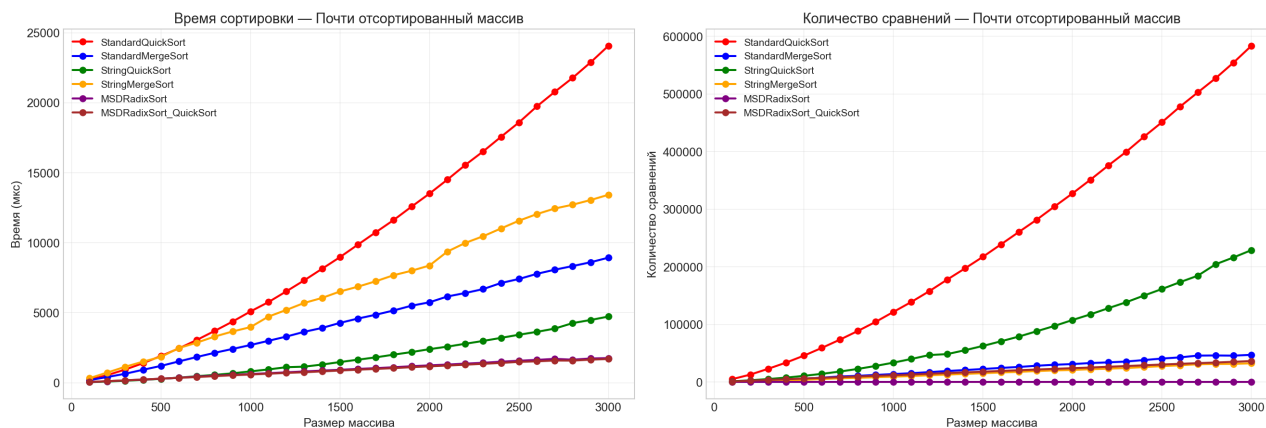
Обратно отсортированный массив



На обратно отсортированном массиве **StandardQuickSort** показал наихудшие результаты как по времени, так и по количеству сравнений. Его графики растут наиболее резко, особенно начиная с размера массива примерно от 1500 элементов. Это объясняется тем, что данный порядок входных данных является неблагоприятным для обычной реализации QuickSort, из-за чего возрастает число сравнений и увеличиваются затраты времени.

Остальные алгоритмы на этом наборе данных работают значительно лучше и располагаются на существенно более низком уровне как по времени, так и по числу сравнений. Разница между ними не столь выражена, как у **StandardQuickSort**.

Почти отсортированный массив



На почти отсортированном массиве снова худший результат показал **StandardQuickSort**. Он заметно уступает остальным алгоритмам как по времени выполнения, так и по числу сравнений, причем разрыв становится особенно заметным на больших размерах массива. Это говорит о высокой чувствительности стандартного QuickSort к структуре входных данных.

По времени **StandardMergeSort** оказался лучше, чем **StringMergeSort**. Наименьшие значения времени показали **MSDRadixSort** и **MSDRadixSort_QuickSort**, что подтверждает эффективность MSD-подхода на почти упорядоченных данных. По числу сравнений **StringQuickSort** оказался немного лучше, чем стандартный QuickSort, однако в целом **StandardMergeSort**, **MSDRadixSort_QuickSort** и **StringMergeSort** имеют близкие значения числа сравнений. Наименьшее число сравнений снова показал **MSDRadixSort**.

Общий вывод

По результатам эксперимента можно сделать вывод, что **специализированные алгоритмы сортировки строк в целом эффективнее стандартных алгоритмов** как по времени, так и по числу посимвольных сравнений. Это связано с тем, что обычные сортировки не учитывают особенности строкового сравнения и могут многократно повторно просматривать одни и те же символы, тогда как строковые алгоритмы используют структуру префиксов и работают с символами более целенаправленно.

Наиболее слабым алгоритмом в большинстве случаев оказался **StandardQuickSort**. Его производительность сильно зависит от порядка входных данных: на обратно отсортированных и почти отсортированных массивах он демонстрирует особенно плохие результаты. Это объясняется тем, что QuickSort чувствителен к выбору опорного элемента и может терять эффективность на неблагоприятно организованных данных.

StandardMergeSort ведет себя стабильнее, чем QuickSort, но все же уступает специализированным методам, поскольку при работе со строками не устраняет избыточные посимвольные сравнения. Среди адаптированных алгоритмов наиболее сильные результаты показали **MSDRadixSort** и **MSDRadixSort_QuickSort**. Их преимущество связано с тем, что

сортировка происходит по символам и эффективно использует разбиение по разрядам, а в гибридной версии дополнительное переключение на QuickSort на малых подмассивах уменьшает накладные расходы.

Таким образом, эксперимент подтверждает, что для сортировки строк предпочтительнее использовать специализированные алгоритмы, особенно если строки имеют общие префиксы или входные данные имеют неблагоприятную структуру.